Implicit Layers

Implicit differentiation, optimization layers nd decision-focused portfolio learning

Yoontae Hwang

June 7, 2026

yoontae.hwang@pusan.ac.kr

Learning goals

By the end of this lecture, you should be able to:

- explain what makes a layer **implicit** rather than explicit;
- use the Implicit Function Theorem to differentiate a solution map;
- derive gradients for unconstrained optimization layers;
- differentiate constrained problems using KKT conditions;
- recognize how decision-focused learning trains models through an optimizer.

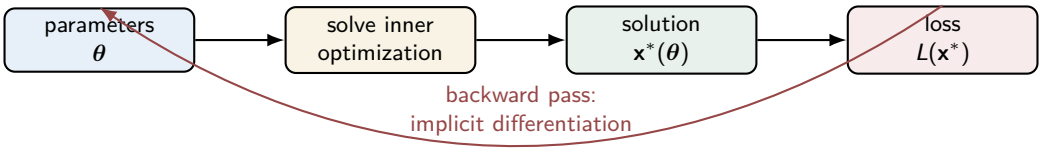
Guiding question

If the output of a layer is obtained by *solving* a problem, how can ordinary backpropagation still train the upstream model?

Big picture: optimization as a layer

Until now, optimization has usually been a **tool**: formulate a problem and solve it.

Here, optimization becomes a **model component**. The layer output is the optimizer of an inner problem.



Core idea

The backward pass does not need to replay every solver iteration. It differentiates the *equation that characterizes the solution*.

Explicit layer vs. implicit layer

Explicit layer

The output is computed by a known formula:

$$\mathbf{y} = f_\theta(\mathbf{x}).$$

Backpropagation follows the computation graph.

Examples:

- affine layer $W\mathbf{x} + b$,
- convolution,
- ReLU or softmax,
- normalization layers.

Implicit layer

The output is defined by an equation:

$$\text{find } \mathbf{y} \text{ such that } g(\mathbf{x}, \mathbf{y}) = \mathbf{0}.$$

A closed-form formula for $\mathbf{y}$ may not exist.

Examples:

- fixed points $\mathbf{y} = f(\mathbf{y}, \mathbf{x})$,
- optimality conditions,
- KKT systems,
- differentiable portfolio decisions.

Three common implicit definitions

1. Fixed-point layer

$$\mathbf{z}^* = f_\theta(\mathbf{z}^*, \mathbf{x}) \quad \Longleftrightarrow \quad F(\mathbf{x}, \mathbf{z}^*) = \mathbf{z}^* - f_\theta(\mathbf{z}^*, \mathbf{x}) = \mathbf{0}.$$

2. Optimization layer

$$\mathbf{x}^*(\boldsymbol{\theta}) = \underset{\mathbf{x}}{\arg\min}\, f(\mathbf{x}, \boldsymbol{\theta}) \quad \Longrightarrow \quad F(\boldsymbol{\theta}, \mathbf{x}^*) = \nabla_{\mathbf{x}} f(\mathbf{x}^*, \boldsymbol{\theta}) = \mathbf{0}.$$

3. Constrained decision layer

$$\mathbf{w}^*(\widehat{\eta}) = \underset{\mathbf{w} \in \mathcal{W}}{\arg\min}\, \Phi(\mathbf{w}; \widehat{\eta}),$$

where the defining equation is usually a KKT system.

Why implicit layers are useful

1. Structure

They can enforce constraints, equilibria, projections, or optimal decisions inside a learning model.

2. Memory efficiency

The backward pass can use local derivatives at the solution instead of storing all solver iterates.

3. Modeling abstraction

The forward pass says *what* is computed. The backward pass uses the equation that defines it.

Key mathematical tool

Implicit Function Theorem

It converts equations of the form $F(a, z) = 0$ into derivatives of the solution map $z^*(a)$.

Companion materials and references

Foundational references

- Kolter, Duvenaud, and Johnson (2020), *Deep Implicit Layers* tutorial.
- Amos and Kolter (2017), *OptNet: Differentiable Optimization as a Layer in Neural Networks*.
- Agrawal, Amos, Barratt, Boyd, Diamond, and Kolter (2019), *Differentiable Convex Optimization Layers*.

Practical library

`cvxpylayers` turns disciplined parametrized convex programs into differentiable PyTorch and Jax.

What will be covered

Mathematical core

- Implicit Function Theorem
- Unconstrained parametric optimization
- Constrained optimization via KKT systems
- Fixed-point layers

Applications

- Optimization as a neural-network layer
- CvxpyLayers
- Bilevel optimization
- Hyperparameter and pricing examples
- Decision-focused portfolio learning

The implicit function problem

Suppose we have an equation

$$F(a, z) = \mathbf{0},$$

where

$$a = (a_1, \ldots, a_p) \in \mathbb{R}^p \quad \text{is a parameter,} \qquad z = (z_1, \ldots, z_n) \in \mathbb{R}^n \quad \text{is the unknown solution.}$$

Implicit definition

Instead of writing an explicit formula $z = f(a)$, the solution is defined by

$$z = z^*(a) \quad \text{such that} \quad F_i(a, z^*(a)) = 0, \qquad i = 1, \ldots, n.$$

Question

How do we compute each scalar sensitivity

$$\frac{\partial z_i^*}{\partial a_j}(a), \qquad i = 1, \ldots, n, \quad j = 1, \ldots, p,$$

without solving for $z^*(a)$ in closed form?

Implicit Function Theorem: scalar derivative form

Theorem

Let $F = (F_1, \dots, F_n) : \mathbb{R}^p \times \mathbb{R}^n \to \mathbb{R}^n$ be continuously differentiable. Suppose that at (a_0, z_0):

1. $F_i(a_0, z_0) = 0$ for all $i = 1, \dots, n$,
2. the matrix with entries
$$M_{i\ell} = \frac{\partial F_i}{\partial z_\ell}(a_0, z_0), \qquad i, \ell = 1, \dots, n,$$
is nonsingular.

Then, locally around a_0, there is a unique differentiable function $z^*(a)$ such that

$$F_i(a, z^*(a)) = 0, \qquad i = 1, \dots, n.$$

For each parameter coordinate a_j, the scalar derivatives satisfy

$$\boxed{\sum_{\ell=1}^{n} \frac{\partial F_i}{\partial z_\ell}(a, z^*(a)) \frac{\partial z_\ell^*}{\partial a_j}(a) + \frac{\partial F_i}{\partial a_j}(a, z^*(a)) = 0, \quad i = 1, \dots, n.}$$

Proof idea: differentiate scalar identities

The formula comes from n scalar identities: $F_i(a, z^*(a)) = 0, \qquad i = 1, \dots, n, \qquad$ for all nearby a.

Step 1: pick one parameter coordinate

Fix a_j. Since each identity is zero,

$$\frac{\partial}{\partial a_j} F_i(a, z^*(a)) = 0, \qquad i = 1, \dots, n.$$

Step 2: apply the chain rule component by component

$$\sum_{\ell=1}^{n} \frac{\partial F_i}{\partial z_\ell}(a, z^*(a)) \frac{\partial z_\ell^*}{\partial a_j}(a) + \frac{\partial F_i}{\partial a_j}(a, z^*(a)) = 0.$$

Step 3: solve for the unknown scalar derivatives

For fixed j, the unknowns are

$$\frac{\partial z_1^*}{\partial a_j}, \dots, \frac{\partial z_n^*}{\partial a_j}.$$

They are obtained by solving the n linear equations above.

IFT dictionary for implicit layers

Setting	Scalar equations	Parameter	Solution
Unconstrained optimization	$\dfrac{\partial f}{\partial x_i}(\mathbf{x}, \boldsymbol{\theta}) = 0, \quad i = 1, \ldots, n$	θ_j	$x_i^*(\boldsymbol{\theta})$
Constrained optimization	stationarity equations plus active constraint equations	θ_j	$x_i^*, \mu_i^*, \lambda_s^*$
Fixed-point layer	$z_i - f_{\boldsymbol{\theta},i}(\mathbf{z}, \mathbf{x}) = 0, \quad i = 1, \ldots, n$	x_j or θ_j	z_i^*
Decision layer	scalar KKT equations for $w_i^*(\widehat{\eta}_\theta)$	component of $\widehat{\eta}_\theta$	portfolio weight w_i^*

Modeling assumption

Nonsingularity often corresponds to strong convexity, a stable active set, strict complementarity, or contraction.

Unconstrained parametric optimization

Start with the simplest optimization layer:

$$\mathbf{x}^*(\boldsymbol{\theta}) = \underset{\mathbf{x}\in\mathbb{R}^n}{\arg\min}\, f(\mathbf{x}, \boldsymbol{\theta}), \qquad \mathbf{x}^* = (x_1^*, \ldots, x_n^*), \quad \boldsymbol{\theta} = (\theta_1, \ldots, \theta_p).$$

Helpful assumptions

- $f(\cdot, \boldsymbol{\theta})$ is strongly convex in $\mathbf{x}$.
- f is twice continuously differentiable.
- The optimizer $\mathbf{x}^*(\boldsymbol{\theta})$ is unique and locally differentiable.

Sensitivity question

How does each optimizer coordinate change when one parameter coordinate changes?

$$\frac{\partial x_i^*}{\partial \theta_j}(\boldsymbol{\theta}), \qquad i = 1, \ldots, n, \quad j = 1, \ldots, p.$$

First-order optimality defines the solution

Because $f(\cdot, \boldsymbol{\theta})$ is differentiable and strongly convex,

$$\mathbf{x}^*(\boldsymbol{\theta}) = \arg\min_{\mathbf{x}} f(\mathbf{x}, \boldsymbol{\theta}) \quad \Longleftrightarrow \quad \frac{\partial f}{\partial x_i}(\mathbf{x}^*(\boldsymbol{\theta}), \boldsymbol{\theta}) = 0, \quad i = 1, \ldots, n.$$

Implicit scalar equations

Define

$$F_i(\boldsymbol{\theta}, \mathbf{x}) = \frac{\partial f}{\partial x_i}(\mathbf{x}, \boldsymbol{\theta}), \qquad i = 1, \ldots, n.$$

Then the optimizer is implicitly defined by

$$F_i(\boldsymbol{\theta}, \mathbf{x}^*(\boldsymbol{\theta})) = 0, \qquad i = 1, \ldots, n.$$

Key idea

Differentiate the **optimality condition**, not the numerical algorithm that found the optimizer.

Component notation for second derivatives

Let

$$\mathbf{x} \in \mathbb{R}^n, \qquad \boldsymbol{\theta} \in \mathbb{R}^p, \qquad F_i(\boldsymbol{\theta}, \mathbf{x}) = \frac{\partial f}{\partial x_i}(\mathbf{x}, \boldsymbol{\theta}).$$

Derivative with respect to x_ℓ

$$\frac{\partial F_i}{\partial x_\ell} = \frac{\partial^2 f}{\partial x_i \partial x_\ell}, \qquad i, \ell = 1, \ldots, n.$$

These entries describe the curvature of f with respect to the optimizer variables.

Derivative with respect to θ_j

$$\frac{\partial F_i}{\partial \theta_j} = \frac{\partial^2 f}{\partial x_i \partial \theta_j}, \qquad i = 1, \ldots, n, \quad j = 1, \ldots, p.$$

These entries describe how the i-th stationarity equation changes with the j-th parameter.

Detailed derivation in scalar form

The stationarity identities are

$$\frac{\partial f}{\partial x_i}(\mathbf{x}^*(\boldsymbol{\theta}), \boldsymbol{\theta}) = 0, \qquad i = 1, \ldots, n.$$

Differentiate with respect to one parameter θ_j

There are two effects:

- an indirect effect through each $x_\ell^*(\boldsymbol{\theta})$,
- a direct effect through θ_j.

Thus, for each $i = 1, \ldots, n$,

$$\sum_{\ell=1}^{n} \frac{\partial^2 f}{\partial x_i \partial x_\ell}(\mathbf{x}^*, \boldsymbol{\theta}) \frac{\partial x_\ell^*}{\partial \theta_j}(\boldsymbol{\theta}) + \frac{\partial^2 f}{\partial x_i \partial \theta_j}(\mathbf{x}^*, \boldsymbol{\theta}) = 0.$$

Interpretation

For fixed j, solve these n equations for $\frac{\partial x_1^*}{\partial \theta_j}, \ldots, \frac{\partial x_n^*}{\partial \theta_j}$. Repeat this for $j = 1, \ldots, p$ if all sensitivities are needed.

Unconstrained sensitivity formula

Theorem

If $f(\cdot, \theta)$ is strongly convex and twice differentiable, then for each parameter coordinate θ_j the scalar sensitivities are determined by

$$\boxed{\sum_{\ell=1}^{n} H_{i\ell} \frac{\partial x_\ell^*}{\partial \theta_j} = -C_{ij}, \qquad i = 1, \ldots, n,}$$

where

$$H_{i\ell} = \frac{\partial^2 f}{\partial x_i \partial x_\ell}(\mathbf{x}^*, \boldsymbol{\theta}), \qquad C_{ij} = \frac{\partial^2 f}{\partial x_i \partial \theta_j}(\mathbf{x}^*, \boldsymbol{\theta}).$$

No explicit inverse in practice

For each j, this means solve a linear system for the unknowns $\frac{\partial x_1^*}{\partial \theta_j}, \ldots, \frac{\partial x_n^*}{\partial \theta_j}$. Numerical implementations should solve the linear equations directly.

Reverse-mode form for backpropagation

In neural networks, we often do not need every scalar derivative

$$\frac{\partial x_i^*}{\partial \theta_j}.$$

We need the gradient of a scalar loss $L(\mathbf{x}^*)$.

Adjoint linear solve in component form

Let

$$g_i = \frac{\partial L}{\partial x_i^*}, \qquad i = 1, \ldots, n.$$

Solve for $u_1, \ldots, u_n$ from

$$\sum_{i=1}^{n} H_{i\ell} u_i = g_\ell, \qquad \ell = 1, \ldots, n.$$

Then, for each parameter coordinate θ_j,

$$\boxed{\frac{\partial L}{\partial \theta_j} = -\sum_{i=1}^{n} C_{ij} u_i.}$$

Example 1: quadratic objective

Consider

$$f(\mathbf{x}, \boldsymbol{\theta}) = \frac{1}{2}\mathbf{x}^\top A\mathbf{x} + \boldsymbol{\theta}^\top \mathbf{x}, \qquad A \succ 0,$$

where $\boldsymbol{\theta} \in \mathbb{R}^n$.

Direct solution

For each i, $\frac{\partial f}{\partial x_i} = \sum_{\ell=1}^n A_{i\ell} x_\ell + \theta_i$. Setting each scalar derivative to zero gives $\sum_{\ell=1}^n A_{i\ell} x_\ell^* + \theta_i = 0, \qquad i = 1, \ldots, n$. Hence $\mathbf{x}^*(\boldsymbol{\theta}) = -A^{-1}\boldsymbol{\theta}$.

Sensitivity by scalar differentiation

Differentiate with respect to θ_j: $\sum_{\ell=1}^n A_{i\ell} \frac{\partial x_\ell^*}{\partial \theta_j} + \delta_{ij} = 0, \qquad i = 1, \ldots, n.$

Therefore

$$\frac{\partial x_i^*}{\partial \theta_j} = -(A^{-1})_{ij}.$$

Example 2: projection-like objective

Let

$$f(\mathbf{x}, \boldsymbol{\theta}) = \frac{1}{2} \|\mathbf{x} - \boldsymbol{\theta}\|_2^2 .$$

Optimizer

For each coordinate,

$$\frac{\partial f}{\partial x_i} = x_i - \theta_i .$$

The stationarity equations are

$$x_i^* - \theta_i = 0, \qquad i = 1, \ldots, n,$$

so

$$x_i^*(\boldsymbol{\theta}) = \theta_i .$$

Sensitivity

For each i, j,

$$\frac{\partial x_i^*}{\partial \theta_j} = \frac{\partial \theta_i}{\partial \theta_j} = \delta_{ij},$$

which matches the obvious solution $\mathbf{x}^*(\boldsymbol{\theta}) = \boldsymbol{\theta}$.

Constrained parametric optimization

Many optimization layers include constraints:

$$P_{\boldsymbol{\theta}}: \qquad \min_{\mathbf{x}} f(\mathbf{x}, \boldsymbol{\theta}) \quad \text{s.t.} \quad \mathbf{g}(\mathbf{x}, \boldsymbol{\theta}) \leq 0, \qquad \mathbf{h}(\mathbf{x}, \boldsymbol{\theta}) = 0.$$

- $\mathbf{g} = (g_1, \ldots, g_m)$ are inequality constraints.
- $\mathbf{h} = (h_1, \ldots, h_r)$ are equality constraints.
- We assume a unique primal solution $\mathbf{x}^*(\boldsymbol{\theta})$.

Goal

Compute scalar sensitivities

$$\frac{\partial x_i^*}{\partial \theta_j}(\boldsymbol{\theta}), \qquad i = 1, \ldots, n, \quad j = 1, \ldots, p.$$

Key idea

Differentiate the KKT conditions rather than only the stationarity equations for f.

KKT conditions

Define the Lagrangian

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}; \boldsymbol{\theta}) = f(\mathbf{x}, \boldsymbol{\theta}) + \sum_{i=1}^{m} \mu_i g_i(\mathbf{x}, \boldsymbol{\theta}) + \sum_{s=1}^{r} \lambda_s h_s(\mathbf{x}, \boldsymbol{\theta}),$$

where $\mu_i \geq 0$ are inequality multipliers and λ_s are equality multipliers.

KKT system at a regular local optimum

$$\begin{aligned}
&\text{stationarity:} && \frac{\partial \mathcal{L}}{\partial x_k}(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*; \boldsymbol{\theta}) = 0, && k = 1, \ldots, n, \\
&\text{primal feasibility:} && g_i(\mathbf{x}^*, \boldsymbol{\theta}) \leq 0, && i = 1, \ldots, m, \\
& && h_s(\mathbf{x}^*, \boldsymbol{\theta}) = 0, && s = 1, \ldots, r, \\
&\text{dual feasibility:} && \mu_i^* \geq 0, && i = 1, \ldots, m, \\
&\text{complementarity:} && \mu_i^* g_i(\mathbf{x}^*, \boldsymbol{\theta}) = 0, && i = 1, \ldots, m.
\end{aligned}$$

Active constraints simplify the derivative

An inequality constraint is **active** if

$$g_i(\mathbf{x}^*, \boldsymbol{\theta}) = 0.$$

Let $\mathcal{A}$ be the active set.

Under strict complementarity

- active constraints have $\mu_i^* > 0$,
- inactive constraints have $g_i(\mathbf{x}^*, \boldsymbol{\theta}) < 0$ and remain inactive locally.

Clean teaching version

Differentiate only:

- stationarity equations $\partial \mathcal{L} / \partial x_k = 0$,
- active inequality equalities $g_i(\mathbf{x}, \boldsymbol{\theta}) = 0$ for $i \in \mathcal{A}$,
- equality constraints $h_s(\mathbf{x}, \boldsymbol{\theta}) = 0$.

Important caveat

If the active set changes, the solution map may be only piecewise differentiable.

Differentiating the active-set KKT system

Fix one parameter coordinate θ_j. Define the unknown scalar sensitivities

$$u_\ell = \frac{\partial x_\ell^*}{\partial \theta_j}, \qquad v_i = \frac{\partial \mu_i^*}{\partial \theta_j} \ (i \in \mathcal{A}), \qquad w_s = \frac{\partial \lambda_s^*}{\partial \theta_j}.$$

All derivatives below are evaluated at $(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*; \boldsymbol{\theta})$.

Scalar linearized KKT equations

$$\sum_{\ell=1}^{n} \frac{\partial^2 \mathcal{L}}{\partial x_k \partial x_\ell} u_\ell + \sum_{i \in \mathcal{A}} \frac{\partial g_i}{\partial x_k} v_i + \sum_{s=1}^{r} \frac{\partial h_s}{\partial x_k} w_s = -\frac{\partial^2 \mathcal{L}}{\partial x_k \partial \theta_j}, \qquad k = 1, \dots, n,$$

$$\sum_{\ell=1}^{n} \frac{\partial g_i}{\partial x_\ell} u_\ell = -\frac{\partial g_i}{\partial \theta_j}, \qquad i \in \mathcal{A},$$

$$\sum_{\ell=1}^{n} \frac{\partial h_s}{\partial x_\ell} u_\ell = -\frac{\partial h_s}{\partial \theta_j}, \qquad s = 1, \dots, r.$$

Backward pass

Differentiating a constrained optimization layer means solving these structured scalar linear equations.

Where the scalar KKT equations come from

Stationarity derivative

For each stationarity equation

$$\frac{\partial \mathcal{L}}{\partial x_k}(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*; \boldsymbol{\theta}) = 0,$$

differentiating with respect to θ_j gives

$$\sum_{\ell=1}^{n} \frac{\partial^2 \mathcal{L}}{\partial x_k \partial x_\ell} \frac{\partial x_\ell^*}{\partial \theta_j} + \sum_{i \in \mathcal{A}} \frac{\partial g_i}{\partial x_k} \frac{\partial \mu_i^*}{\partial \theta_j} + \sum_{s=1}^{r} \frac{\partial h_s}{\partial x_k} \frac{\partial \lambda_s^*}{\partial \theta_j} + \frac{\partial^2 \mathcal{L}}{\partial x_k \partial \theta_j} = 0.$$

Active inequality and equality derivatives

$$\sum_{\ell=1}^{n} \frac{\partial g_i}{\partial x_\ell} \frac{\partial x_\ell^*}{\partial \theta_j} + \frac{\partial g_i}{\partial \theta_j} = 0, \qquad i \in \mathcal{A},$$

$$\sum_{\ell=1}^{n} \frac{\partial h_s}{\partial x_\ell} \frac{\partial x_\ell^*}{\partial \theta_j} + \frac{\partial h_s}{\partial \theta_j} = 0, \qquad s = 1, \ldots, r.$$

When is the scalar KKT system nonsingular?

The Implicit Function Theorem requires the linearized scalar KKT equations to have a unique solution for the sensitivities.

Typical sufficient conditions

- Strong second-order sufficient condition for the Lagrangian Hessian.
- LICQ: active constraint gradients are linearly independent.
- Strict complementarity for active inequalities.
- A locally stable active set.

Why this matters

At a kink, such as when a constraint switches from inactive to active, the classical derivative may fail to exist.

Practical note

Modern differentiable solvers may return generalized, approximate, or regularized derivatives even when classical assumptions are imperfect.

Example 1: equality-constrained QP

Consider

$$\min_{\mathbf{x}} \ \frac{1}{2}\mathbf{x}^\top P\mathbf{x} + \mathbf{c}^\top \mathbf{x} \qquad \text{s.t.} \qquad A\mathbf{x} = \mathbf{b}(\boldsymbol{\theta}),$$

with $P \succ 0$.

KKT equations in components

For $i = 1, \ldots, n$ and $s = 1, \ldots, r$,

$$\sum_{\ell=1}^{n} P_{i\ell} x_\ell + c_i + \sum_{s=1}^{r} A_{si} \lambda_s = 0,$$

$$\sum_{i=1}^{n} A_{si} x_i = b_s(\boldsymbol{\theta}).$$

Equality-constrained QP: derivative

Differentiate the KKT equations with respect to one parameter coordinate θ_j:

$$\sum_{\ell=1}^{n} P_{i\ell} \frac{\partial x_\ell^*}{\partial \theta_j} + \sum_{s=1}^{r} A_{si} \frac{\partial \lambda_s^*}{\partial \theta_j} = 0, \qquad i = 1, \ldots, n,$$

$$\sum_{i=1}^{n} A_{si} \frac{\partial x_i^*}{\partial \theta_j} = \frac{\partial b_s}{\partial \theta_j}(\boldsymbol{\theta}), \qquad s = 1, \ldots, r.$$

If $\mathbf{b}(\theta) = \theta b_0$ is scalar-parameterized

Then

$$\frac{\partial b_s}{\partial \theta} = (b_0)_s,$$

so the sensitivity equations become

$$\sum_{\ell=1}^{n} P_{i\ell} \frac{\mathrm{d}x_\ell^*}{\mathrm{d}\theta} + \sum_{s=1}^{r} A_{si} \frac{\mathrm{d}\lambda_s^*}{\mathrm{d}\theta} = 0,$$

$$\sum_{i=1}^{n} A_{si} \frac{\mathrm{d}x_i^*}{\mathrm{d}\theta} = (b_0)_s.$$

Example 2: projection onto an interval

Consider the scalar constrained problem

$$x^*(\theta) = \arg\min_{0 \le x \le 1} \frac{1}{2}(x - \theta)^2.$$

The solution is the clipped value

$$x^*(\theta) = \min\{1, \max\{0, \theta\}\}.$$

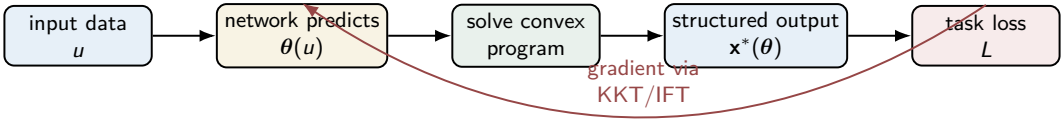
Derivative away from kinks

$$\frac{\mathrm{d}x^*}{\mathrm{d}\theta} = \begin{cases} 0, & \theta < 0, \\ 1, & 0 < \theta < 1, \\ 0, & \theta > 1. \end{cases}$$

At $\theta = 0$ or $\theta = 1$

The active set changes, so the classical derivative is not unique. This is why active-set stability matters.

Optimization-as-a-layer pipeline



Forward pass

Solve an optimization problem to produce $\mathbf{x}^*(\theta)$.

Backward pass

Differentiate the scalar optimality conditions that characterize $\mathbf{x}^*(\theta)$.

Why not simply backpropagate through the solver?

A numerical solver may run many iterations: $\mathbf{x}^0 \to \mathbf{x}^1 \to \cdots \to \mathbf{x}^T \approx \mathbf{x}^*$.

Unrolling drawbacks

- high memory cost: store all iterates;
- gradients depend on solver implementation and stopping time;
- slow when the inner optimization takes many iterations.

Implicit differentiation alternative

Once the solver returns $\mathbf{z}^*$, use scalar equations

$$G_i(\mathbf{z}^*, \boldsymbol{\theta}) = 0, \qquad i = 1, \dots, q.$$

For each parameter coordinate θ_j,

$$\sum_{\ell=1}^{q} \frac{\partial G_i}{\partial z_\ell}(\mathbf{z}^*, \boldsymbol{\theta}) \frac{\partial z_\ell^*}{\partial \theta_j} + \frac{\partial G_i}{\partial \theta_j}(\mathbf{z}^*, \boldsymbol{\theta}) = 0, \qquad i = 1, \dots, q.$$

The backward pass depends on the solution and local scalar derivatives, not on solver history.

CvxpyLayers: differentiable convex optimization

`cvxpylayers` turns a CVXPY problem into a differentiable PyTorch, JAX, or TensorFlow layer.

Forward pass

Given parameter values, solve a convex optimization problem.

Backward pass

Differentiate through the scalar KKT conditions of the cone-program representation.

Important modeling rule

The CVXPY problem must satisfy **disciplined parametrized programming** (DPP). A common pitfall is using a parameterized matrix inside `quad_form(x, P)`.

CvxpyLayers example: projection onto the simplex

```
import cvxpy as cp
from cvxpylayers.torch import CvxpyLayer
import torch

# Differentiable problem:
#   minimize_x 0.5 ||x - theta||_2^2
#   subject to x >= 0, sum(x) = 1
n = 5
x = cp.Variable(n)
theta = cp.Parameter(n)

objective = cp.Minimize(0.5 * cp.sum_squares(x - theta))
constraints = [x >= 0, cp.sum(x) == 1]
prob = cp.Problem(objective, constraints)
assert prob.is_dpp()

layer = CvxpyLayer(prob, parameters=[theta], variables=[x])

theta_t = torch.randn(n, requires_grad=True)
x_star, = layer(theta_t)

loss = downstream_loss(x_star)
loss.backward()
```

CvxpyLayers example: what it teaches

Forward pass

The layer returns the Euclidean projection of θ onto the probability simplex:

$$\Delta_n = \{x \in \mathbb{R}^n : x_i \geq 0,\ \mathbf{1}^\top x = 1\}.$$

Backward pass

Autodiff receives scalar derivatives of the projection map, such as

$$\frac{\partial x_i^*}{\partial \theta_j}.$$

Locally, these derivatives depend on which simplex constraints are active.

Student-friendly interpretation

The layer turns unconstrained scores θ into valid portfolio weights or probabilities x^*.

Cone-program KKT sketch

Many convex programs are canonicalized into cone form:

$$\min_{\mathbf{x}} \ \mathbf{c}^\top \mathbf{x} \qquad \text{s.t.} \qquad A\mathbf{x} + \mathbf{s} = \mathbf{b}, \quad \mathbf{s} \in \mathcal{K}.$$

A schematic KKT system is

$$\begin{aligned} A\mathbf{x} + \mathbf{s} &= \mathbf{b}, \\ A^\top \mathbf{y} + \mathbf{c} &= 0, \\ \mathbf{s} \in \mathcal{K}, \qquad \mathbf{y} &\in \mathcal{K}^*, \\ \langle \mathbf{s}, \mathbf{y} \rangle &= 0, \end{aligned}$$

up to the sign convention chosen for the dual variable.

Differentiable cone solving

Parameters can enter A, $\mathbf{b}$, and $\mathbf{c}$. For a scalar parameter ρ such as an entry A_{st}, b_s, or c_i, differentiating the scalar KKT equations gives sensitivities such as

$$\frac{\partial x_i}{\partial \rho}, \qquad \frac{\partial s_i}{\partial \rho}, \qquad \frac{\partial y_i}{\partial \rho}.$$

The cone solver backward pass computes these sensitivities by solving the corresponding linearized KKT equations.

From prediction-first to decision-focused learning

Classical financial machine learning often uses a two-stage pipeline:

$$\text{features} \longrightarrow \widehat{\text{return}} \longrightarrow \text{portfolio optimizer}.$$

The forecasting model is trained to minimize prediction error, such as MSE.

Problem: objective mismatch

A small prediction error does not necessarily imply a good portfolio decision:

$$\text{good forecast metric} \not\Longrightarrow \text{good realized utility, regret, or tail risk}.$$

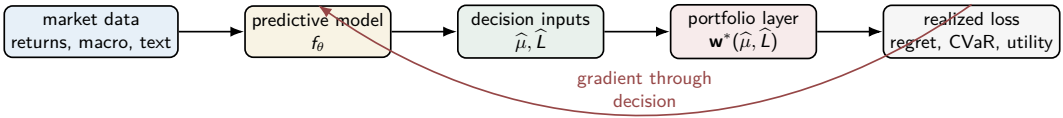
The optimizer may amplify errors in exactly the assets that matter for allocation.

Decision-focused idea

Train the upstream model through the downstream decision layer:

$$\min_{\theta} \; \mathcal{L}_{\text{decision}}\left(\mathbf{w}^*(\widehat{\eta}_\theta), \text{realized market outcome}\right).$$

Decision-focused computational graph



Chain rule through the decision

If η_θ is predicted by a model and $\mathbf{w}^*(\eta)$ solves an inner problem, then

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial \mathbf{w}^*} \frac{\partial \mathbf{w}^*}{\partial \eta} \frac{\partial \eta_\theta}{\partial \theta}.$$

Portfolio optimizer as a decision layer

A common inner layer maps predicted moments to a long-only portfolio:

$$\begin{aligned} \mathbf{w}^*(\widehat{\boldsymbol{\mu}}, \widehat{L}) = \arg\min_{\mathbf{w}, s} \quad & \lambda s - \widehat{\boldsymbol{\mu}}^\top \mathbf{w} \\ \text{s.t.} \quad & \left\| \widehat{L}\mathbf{w} \right\|_2 \le s, \qquad s \ge 0, \\ & \mathbf{1}^\top \mathbf{w} = 1, \qquad 0 \le w_i \le 1. \end{aligned}$$

Here $\widehat{\boldsymbol{\Sigma}} = \widehat{L}^\top \widehat{L}$, so $\left\| \widehat{L}\mathbf{w} \right\|_2$ is the predicted portfolio standard deviation.

Why this is an implicit layer

The output $\mathbf{w}^*$ is defined as the optimizer of a convex problem. The backward pass differentiates its KKT system.

Decision regret: train for the downstream objective

Suppose $\widehat{\mathbf{w}}_{t+h}$ is chosen using predicted quantities, while $\mathbf{w}^{\star}_{t+h}$ is an ex-post benchmark computed from realized future moments.

Realized risk-return costs

$$J^{\star}_{t+h} = \lambda \left\| L^{\star}_{t+h} \mathbf{w}^{\star}_{t+h} \right\|_2 - (\mu^{\star}_{t+h})^{\top} \mathbf{w}^{\star}_{t+h},$$
$$\widehat{J}_{t+h} = \lambda \left\| L^{\star}_{t+h} \widehat{\mathbf{w}}_{t+h} \right\|_2 - (\mu^{\star}_{t+h})^{\top} \widehat{\mathbf{w}}_{t+h}.$$

Decision-focused loss

$$\Delta J_{t+h} = \widehat{J}_{t+h} - J^{\star}_{t+h}, \qquad \mathcal{L}_{\text{Decision}} = \frac{1}{NH} \sum_{h=1}^{H} |\Delta J_{t+h}|.$$

Decision-focused gradient: the important path

Because $J^\star$ is computed using benchmark future quantities, it is constant with respect to model parameters θ. The gradient flows through $\widehat{J}$ and through the optimization layer.

Gradient with respect to portfolio weights

For

$$\widehat{J} = \lambda \left\| L^\star \widehat{\mathbf{w}} \right\|_2 - (\mu^\star)^\top \widehat{\mathbf{w}},$$

we have, when $L^\star \widehat{\mathbf{w}} \neq 0$,

$$\nabla_{\widehat{\mathbf{w}}} \widehat{J} = \lambda \frac{(L^\star)^\top L^\star \widehat{\mathbf{w}}}{\left\| L^\star \widehat{\mathbf{w}} \right\|_2} - \mu^\star.$$

Then use implicit sensitivity

The optimizer supplies $\partial \widehat{\mathbf{w}} / \partial \widehat{\boldsymbol{\mu}}$ and $\partial \widehat{\mathbf{w}} / \partial \widehat{L}$ through its KKT system.

Markowitz weights: a clean closed-form case

To see the KKT sensitivity clearly, ignore box constraints and consider

$$\min_{\mathbf{w}} \ \frac{1}{2}\mathbf{w}^\top \widehat{\boldsymbol{\Sigma}}\mathbf{w} - \widehat{\boldsymbol{\mu}}^\top \mathbf{w} \qquad \text{s.t.} \qquad \mathbf{1}^\top \mathbf{w} = 1.$$

KKT conditions

$$\widehat{\boldsymbol{\Sigma}}\mathbf{w} - \widehat{\boldsymbol{\mu}} + \gamma \mathbf{1} = 0, \qquad \mathbf{1}^\top \mathbf{w} = 1.$$

Solving gives

$$\mathbf{w}^*(\widehat{\boldsymbol{\mu}}) = \widehat{\boldsymbol{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}} - \widehat{\boldsymbol{\Sigma}}^{-1}\mathbf{1} \cdot \frac{\mathbf{1}^\top \widehat{\boldsymbol{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}} - 1}{\mathbf{1}^\top \widehat{\boldsymbol{\Sigma}}^{-1}\mathbf{1}}.$$

Sensitivity of Markowitz weights to predicted returns

Differentiate the closed-form solution with respect to $\widehat{\boldsymbol{\mu}}$:

$$\boxed{\frac{\partial \mathbf{w}^*}{\partial \widehat{\boldsymbol{\mu}}} = \widehat{\boldsymbol{\Sigma}}^{-1} - \frac{\widehat{\boldsymbol{\Sigma}}^{-1}\mathbf{1}\mathbf{1}^\top \widehat{\boldsymbol{\Sigma}}^{-1}}{\mathbf{1}^\top \widehat{\boldsymbol{\Sigma}}^{-1}\mathbf{1}}.}$$

Interpretation

The matrix adjusts return shocks while preserving the budget constraint:

$$\mathbf{1}^\top \mathrm{d}\mathbf{w} = 0.$$

A higher predicted return for one asset must shift weight away from other assets.

Numerical portfolio example: two assets

Let

$$\widehat{\boldsymbol{\mu}} = \begin{pmatrix} 0.08 \\ 0.06 \end{pmatrix}, \qquad \widehat{\boldsymbol{\Sigma}} = \begin{pmatrix} 0.04 & 0 \\ 0 & 0.09 \end{pmatrix}.$$

Then

$$\widehat{\boldsymbol{\Sigma}}^{-1} = \begin{pmatrix} 25 & 0 \\ 0 & 11.11 \end{pmatrix}.$$

Full-investment Markowitz solution

$$\mathbf{w}^* \approx \begin{pmatrix} 0.846 \\ 0.154 \end{pmatrix}.$$

Most weight goes to asset 1 because it has higher predicted return and lower variance.

Return sensitivity

$$\frac{\partial \mathbf{w}^*}{\partial \widehat{\boldsymbol{\mu}}} \approx \begin{pmatrix} 7.69 & -7.69 \\ -7.69 & 7.69 \end{pmatrix}.$$

A 0.01 increase in asset 1's predicted return shifts about 7.7% weight from asset 2 to asset 1.

SIT: why signatures for asset allocation?

The Signature-Informed Transformer (SIT) uses path signatures as features for risk-aware allocation.

Path signature idea

For a path $\mathbf{X} : [s, t] \to \mathbb{R}^d$, the truncated signature is

$$\mathrm{Sig}^M(\mathbf{X}_{[s,t]}) = \left(1, \int_s^t \mathrm{d}\mathbf{X}_u, \int_s^t \int_s^u \mathrm{d}\mathbf{X}_r \otimes \mathrm{d}\mathbf{X}_u, \dots\right).$$

Financial interpretation

- First-order terms capture net increments.
- Second-order cross terms capture lead-lag and cross-asset effects.
- Chen's relation supports efficient updates over time windows.

SIT: decision-focused but not necessarily implicit

At decision time t_i, SIT maps financial information to multi-step return logits:

$$g_\theta : \mathcal{F}_{t_i} \longmapsto \widehat{\boldsymbol{\mu}}_{t_i}^{1:K} \in \mathbb{R}^{K \times d}.$$

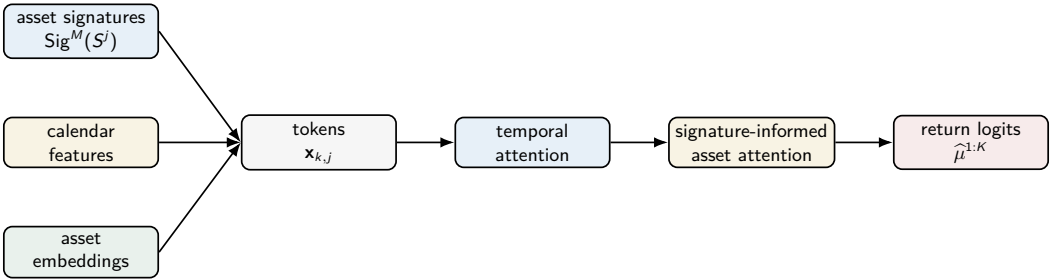
Weights may be produced by a softmax allocation layer:

$$\mathbf{w}_{t_i}^{(k)}(\theta) = \mathrm{softmax}\left(\frac{\widehat{\boldsymbol{\mu}}_{t_i}^{(k)}(\theta)}{\tau} \right), \qquad \mathbf{w}_{t_i}^{(k)} \in \Delta_d.$$

Clarification

The softmax allocation is an **explicit** differentiable layer. SIT still illustrates decision-focused learning because it trains using realized portfolio loss rather than only forecast error.

SIT architecture: what is signature-informed?



Token construction

$$\mathbf{x}_{k,j} = W_{\mathrm{proj}} \left[\mathbf{e}_{\mathrm{sig},k,j} \oplus \mathbf{e}_{\mathrm{date},k} \oplus \mathbf{e}^{j}_{\mathrm{asset}} \right].$$

Signature-informed attention: relational bias

For a fixed time slice k and attention head h, standard asset attention uses

$$\frac{\mathbf{Q}_{k,h}\mathbf{K}_{k,h}^{\top}}{\sqrt{d_k}}.$$

SIT adds a signature-based relational bias:

$$\mathrm{Logits}_{k,h} = \frac{\mathbf{Q}_{k,h}\mathbf{K}_{k,h}^{\top}}{\sqrt{d_k}} + \gamma \mathbf{B}_{k,h}, \qquad \gamma = \mathrm{softplus}(\widehat{\gamma}) > 0.$$

Dynamic bias for asset pair (j, l)

$$\beta_{i,j,l} = \mathrm{MLP}_{\beta}(\mathbf{c}_{i,j,l}), \qquad \mathbf{q}_{k,j}^{\mathrm{dyn}} = \mathrm{MLP}_{q}(\mathbf{x}'_{k,j}),$$

$$b_{k,h,j,l} = \left\langle (\mathbf{q}_{k,j}^{\mathrm{dyn}})_h, (\beta_{i,j,l})_h \right\rangle.$$

Why the signature bias changes attention

Let

$$z_{j,m} = \frac{(\mathbf{Q}\mathbf{K}^\top)_{j,m}}{\sqrt{d_k}} + \gamma \langle \mathbf{q}_j, \boldsymbol{\beta}_{j,m} \rangle, \qquad \alpha_{j,m} = \frac{e^{z_{j,m}}}{\sum_r e^{z_{j,r}}}.$$

Directional derivative

If $0 < \alpha_{j,l} < 1$, $\gamma > 0$, and $\mathbf{q}_j \neq 0$, then perturbing $\boldsymbol{\beta}_{j,l}$ in direction $\mathbf{q}_j$ gives

$$D^{(\boldsymbol{\beta})}_{\mathbf{q}_j} \alpha_{j,l} = \gamma \alpha_{j,l}(1 - \alpha_{j,l}) \, \|\mathbf{q}_j\|_2^2 > 0.$$

Interpretation

If the relational signature feature aligns with the query asset's information need, the attention weight on that asset pair increases.

SIT training: CVaR as a decision loss

For each scenario ω, SIT observes a sequence of K realized portfolio losses

$$L_\omega^{(1)}(\theta), \ldots, L_\omega^{(K)}(\theta).$$

A risk-aware objective can minimize expected tail risk:

$$\min_\theta \ \mathbb{E}_{\omega \sim \mathcal{D}} \left[\mathrm{CVaR}_\alpha \left(\{ L_\omega^{(k)}(\theta) \}_{k=1}^K \right) \right].$$

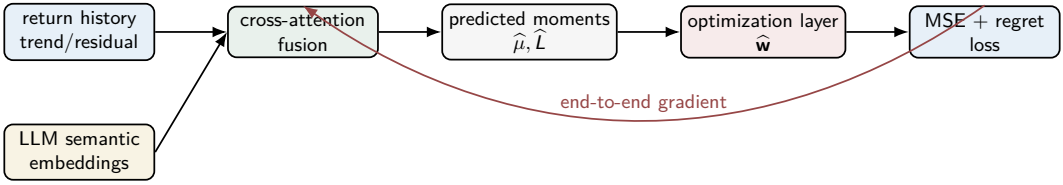
Rockafellar-Uryasev representation

For a random loss Z,

$$\mathrm{CVaR}_\alpha(Z) = \min_{\nu \in \mathbb{R}} \left\{ \nu + \frac{1}{1-\alpha} \mathbb{E}[(Z-\nu)^+] \right\}.$$

DINN: unified forecasting and portfolio selection

Decision-Informed Neural Networks (DINN) combine multi-modal representation learning with a differentiable portfolio optimization layer.



Main principle

Forecasting is not separate from portfolio selection. The optimizer makes the prediction module aware of how its outputs will be used.

DINN inputs: numerical and semantic context

DINN builds structured inputs from two sources.

Numerical return features

Normalize returns over a lookback window and decompose them into trend and residual components:

$$r'_{t,i} = \frac{r_{t,i} - \mu_{t,i}}{\sigma_{t,i}}, \qquad r'_{t,i} = \tau_{t,i} + \rho_{t,i}.$$

Trend τ captures slow movement; residual ρ captures short-term fluctuations.

LLM-enhanced semantic embeddings

Text prompts summarize asset relationships, sector context, and macro variables. The prompt embeddings become semantic inputs for the forecasting module.

DINN cross-attention: fusing modalities

Let $\mathbf{T}_t$ and $\mathbf{R}_t$ denote trend and residual matrices.

Two cross-attention streams

$$\mathbf{C}_{\text{market}} = \text{CrossAttn}(\mathbf{T}_t, E_{\text{macro}}), \qquad \mathbf{C}_{\text{stock}} = \text{CrossAttn}(\mathbf{R}_t, E_{\text{stocks}}).$$

Sparse attention idea

For large textual embeddings, full attention is expensive. Prob-sparse attention keeps only a sampled subset $\mathcal{S}(i)$ of relevant keys:

$$\alpha_{i,j} = 0 \quad (j \notin \mathcal{S}(i)), \qquad \alpha_{i,j} \propto \exp\left(\frac{Q_i K_j^\top}{\sqrt{d_b}}\right) \quad (j \in \mathcal{S}(i)).$$

DINN optimization layer

For each horizon h, the optimization layer solves

$$\begin{aligned} \min_{\mathbf{w}_{t+h}, s_{t+h}} \quad & \lambda s_{t+h} - \widehat{\mu}_{t+h}^{\top} \mathbf{w}_{t+h} \\ \text{s.t.} \quad & \left\| \widehat{L}_{t+h} \mathbf{w}_{t+h} \right\|_2 \leq s_{t+h}, \qquad s_{t+h} \geq 0, \\ & \mathbf{1}^{\top} \mathbf{w}_{t+h} = 1, \qquad 0 \leq w_{t+h,i} \leq 1. \end{aligned}$$

Why it fits this lecture

This is a constrained implicit layer:

$$G(\mathbf{w}^*, s^*, \text{dual variables}; \widehat{\mu}, \widehat{L}) = 0.$$

Differentiating $G = 0$ gives gradients for the upstream neural network.

DINN training objective: prediction plus regret

DINN combines a forecasting loss and a decision loss:

$$\mathcal{L}_{\text{total}} = \beta \mathcal{L}_{\text{MSE}} + (1-\beta)\mathcal{L}_{\text{Decision}}, \qquad \beta \in [0,1].$$

Forecasting term

$$\mathcal{L}_{\text{MSE}} = \frac{1}{NH} \sum_{h=1}^{H} \|\widehat{r}_{t+h} - r_{t+h}\|_2^2 .$$

This improves calibration of predicted returns.

Decision term

$$\mathcal{L}_{\text{Decision}} = \frac{1}{NH} \sum_{h=1}^{H} |\widehat{J}_{t+h} - J^{\star}_{t+h}|.$$

This penalizes realized regret.

SIT and DINN through the decision-focused lens

Model	Representation	Decision-focused training signal
SIT	Path signatures, calendar features, asset embeddings, signature-informed attention	CVaR of realized multi-step portfolio losses, often with explicit softmax allocation weights
DINN	Time-series trend/residual features plus LLM semantic embeddings fused by cross-attention	Hybrid loss: forecast MSE plus decision regret through a constrained portfolio optimization layer

Common principle

Both approaches train the network using the quality of the final portfolio decision, not only the accuracy of intermediate forecasts.

Summary table

Layer type	**Forward equation**	**Backward equation**
Unconstrained optimization	$\nabla_{\mathbf{x}} f(\mathbf{x}^*, \boldsymbol{\theta}) = 0$	$D_{\boldsymbol{\theta}} \mathbf{x}^* = -[\nabla^2_{\mathbf{xx}} f]^{-1} \nabla^2_{\mathbf{x}\boldsymbol{\theta}} f$
Constrained optimization	KKT residual $G(\mathbf{z}^*, \boldsymbol{\theta}) = 0$	$D_{\boldsymbol{\theta}} \mathbf{z}^* = -[G_{\mathbf{z}}]^{-1} G_{\boldsymbol{\theta}}$
Fixed point	$\mathbf{z}^* = f_{\boldsymbol{\theta}}(\mathbf{z}^*, \mathbf{x})$	solve $(I - J_z)^\top w = v$ for reverse-mode gradients
Portfolio decision layer	$\mathbf{w}^* = \arg\min_{\mathbf{w} \in \mathcal{W}} \Phi(\mathbf{w}; \widehat{\eta_\theta})$	differentiate KKT/solver map so realized regret or CVaR trains θ

References

Implicit layers and differentiable optimization

- Kolter, Duvenaud, and Johnson (2020). *Deep Implicit Layers: Neural ODEs, Equilibrium Models, and Beyond*. NeurIPS tutorial. implicit-layers-tutorial.org
- Amos and Kolter (2017). *OptNet: Differentiable Optimization as a Layer in Neural Networks*. ICML.
- Agrawal, Amos, Barratt, Boyd, Diamond, and Kolter (2019). *Differentiable Convex Optimization Layers*. NeurIPS.
- Gould, Hartley, and Campbell (2019). *Deep Declarative Networks*.

Decision-focused portfolio learning examples

- Hwang and Zohren (2026). *Signature-Informed Transformer for Asset Allocation*. ICML
- Hwang, Kong, Zohren, and Lee (2025). *Decision-informed Neural Networks with Large Language Model Integration for Portfolio Optimization*. arXiv:2502.00828.
- Rockafellar and Uryasev (2000). Optimization of conditional value-at-risk.

Looking ahead

Implicit layers turn optimization theory into a differentiable modeling tool.

Main lesson

A solver can appear inside a neural network as long as its output is characterized by differentiable optimality conditions.

What this enables

- end-to-end learning with constraints,
- structured prediction modules,
- bilevel training and hyperparameter optimization,
- decision-focused portfolio learning,
- differentiable risk-aware optimization layers.

Unifying principle

Forward: solve.	Backward: differentiate the defining equation.